

FastAPI Interview Questions for Senior Backend Engineers

Executive Summary: FastAPI is a modern, high-performance web framework for Python, built on Starlette and Pydantic. It leverages Python 3.6+ type hints for automatic validation and generates OpenAPI/JSON Schema documentation [4†L277-L286] [6†L482-L490]. Benchmarks show FastAPI (on Uvicorn) is one of the fastest Python frameworks available [34†L242-L245]. This report compiles 100 interview questions, organized by category, targeting a senior backend engineer (6+ years experience). For each question, we list difficulty, key answer points, and whether to use live coding or discussion. The categories include core concepts, Pydantic/validation, dependency injection, async/concurrency, performance, security, testing, deployment/DevOps, database/ORMs, WebSockets/tasks, API design/versioning, error handling/logging, migration/compatibility, real-world scenarios, and code review. Sources from the official FastAPI docs and related material are cited throughout. The report concludes with a table summarizing question counts per category and a suggested 90-minute interview structure with time allocation per topic.

Core FastAPI Concepts

- **What is FastAPI and how does it differ from Flask or Django?**

- *Difficulty:* Medium.

- *Key Points:*

- FastAPI is an ASGI framework (built on Starlette) for Python 3.6+ focusing on speed and type safety [4†L277-L286].
- It uses Python type hints and Pydantic for data validation, unlike Flask/Django which are WSGI and often use manual validation [4†L277-L286] [6†L482-L490].
- Provides automatic interactive API docs (Swagger UI, ReDoc) out of the box [4†L277-L286].
- High performance: benchmarks rank FastAPI near the top among Python frameworks [34†L242-L245].

- *Recommended:* Conceptual discussion.

- **Describe FastAPI's automatic API documentation (OpenAPI/Swagger). How is it generated?**

- *Difficulty:* Medium.

- *Key Points:*

- FastAPI automatically generates an OpenAPI schema from your path operation definitions and Pydantic models.
- Interactive docs are available at `/docs` (Swagger UI) and `/redoc` by default [4†L277-L286].
- The OpenAPI JSON is exposed (e.g. `/openapi.json`), which clients or tools can consume.
- No extra code needed; customization via `app.openapi()` or doc parameters.

- *Recommended:* Discussion.

- **Explain the FastAPI request/response lifecycle (ASGI, Starlette, Uvicorn).**

- *Difficulty:* Advanced.

- *Key Points:*

- FastAPI apps run on an ASGI server (commonly Uvicorn). Uvicorn provides the event loop and HTTP handling. FastAPI (via Starlette) handles routing and middleware.
- When a request arrives, Uvicorn's event loop invokes FastAPI's router. Dependencies are resolved first, then path operation code is executed.
- Pydantic models validate the request data before invoking the endpoint; the response is returned to Starlette, then Uvicorn to the client.
- FastAPI can mix `async def` and normal `def` endpoints; it runs async endpoints on the event loop and sync endpoints in a threadpool automatically [24†L328-L332] .
- FastAPI's speed partly comes from Starlette + Uvicorn's efficiency [34†L242-L245] [24†L328-L332] .

- *Recommended:* Whiteboard discussion (draw flow).

- **What is the FastAPI CLI and how do you run an app in production?**

- *Difficulty:* Medium.

- *Key Points:*

- The `fastapi` CLI command (e.g. `fastapi run app/main.py`) is a wrapper around Uvicorn that simplifies running the app (port, workers, reload, proxy headers).
- Always use the **exec form** (`CMD ["fastapi", "run", ...]` in Docker) to allow proper signal handling and graceful shutdown [21†L579-L588] [21†L593-L601] .
- You can specify options like `--port 80`, `--workers N` for multi-process, `--reload` for development, and `--proxy-headers` if behind a reverse proxy [21†L579-L588] [77†L7-L14] .
- Example: `fastapi run app/main.py --port 80 --workers 4 --proxy-headers` [21†L579-L588] [22†L61-L69] .

- *Recommended:* Short explanation (no coding).

- **What are middleware in FastAPI and how do you implement one?**

- *Difficulty:* Advanced.

- *Key Points:*

- Middleware is a function that intercepts *every* request before routing and every response before sending. It can inspect/modify requests or responses [97†L249-L258] .
- Use `@app.middleware("http")` decorator. The function receives `request` and a `call_next` function. It can run code before calling `await call_next(request)`, then modify the `response` before returning it [97†L274-L283] [97†L294-L301] .
- Example: timing middleware that adds an `X-Process-Time` header [97†L294-L301] .
- Order matters: multiple middlewares run in declaration order (first to last) before request, and reverse after response.

- *Recommended:* Whiteboard (write example code, see [97†L294-L301]).

• How do startup and shutdown events work in FastAPI?

• *Difficulty:* Advanced.

• *Key Points:*

- FastAPI supports lifespan events via `@app.on_event("startup")` and `@app.on_event("shutdown")`. These run before the first request and after the last response, respectively.
- Common use cases: connect to databases, prepare caches on startup; close connections on shutdown.
- These are asynchronous-safe and integrate with Uvicorn's startup/shutdown signals.
- In Docker, the **exec** form CMD is crucial to trigger these events on container stop [\[21†L579-L588\]](#).

• *Recommended:* Discussion, possibly code snippet.

• How can you mount sub-applications or WSGI apps in FastAPI?

• *Difficulty:* Advanced.

• *Key Points:*

- FastAPI (via Starlette) allows mounting sub-apps using `app.mount("/subpath", other_app)`. The other app can be another Starlette/FastAPI app or even a WSGI app (Flask/Django) wrapped via `starlette.wsgi.WSGIMiddleware`.
- This is useful to split services or include legacy apps.
- Mounting an app preserves routing under the prefix. Example: `app.mount("/legacy", WSGIApp)` to serve a Flask app at `/legacy/*`.
- Note: middleware and dependencies are per app unless designed to share.

• *Recommended:* Discussion.

Pydantic and Data Validation

• How does FastAPI use Pydantic models for request validation and serialization?

• *Difficulty:* Medium.

• *Key Points:*

- Any request body declared as a Pydantic `BaseModel` causes FastAPI to parse and validate the incoming JSON against that model. Field types (e.g. `int`, `str`, custom types) enforce type and constraints.
- Pydantic automatically generates JSON schema for OpenAPI.
- Response models (`response_model`) also use Pydantic to validate and serialize outgoing data.
- All Pydantic features (nested models, custom types, validators) are fully supported [\[6†L482-L490\]](#). No new schema language is needed [\[6†L482-L490\]](#).

• *Recommended:* Discussion (show simple model example).

• How can you add custom validation logic to a Pydantic model?

• *Difficulty:* Advanced.

- *Key Points:*

- Use Pydantic's validators: in v1 use `@validator`, in v2 use `@field_validator`. For example, `@validator('username')` to enforce alphanumeric usernames.
- You can also use `@root_validator` or new v2 features to enforce cross-field checks.
- Define custom data types (by subclassing and implementing `__get_validators__`).
- FastAPI itself doesn't change this mechanism; refer to Pydantic docs (note: `@validator` is deprecated in v2 in favor of `@field_validator` [\[58†L54-L59\]](#)).

- *Recommended:* Whiteboard or code writing (write a Pydantic validator).

- **Explain FastAPI's `response_model` and related options (e.g., `response_model_exclude_unset`).**

- *Difficulty:* Advanced.

- *Key Points:*

- `response_model=YourModel` tells FastAPI to use that Pydantic model for the output. It validates and serializes the return value to JSON based on the model's schema.
- Parameters like `response_model_exclude_unset=True` can omit default/unset fields from the output [\[60†L826-L833\]](#) . Similarly, `exclude_none` omits `None` fields.
- `response_model_include` and `response_model_exclude` allow picking specific fields.
- Setting `response_model=None` disables model validation for the response [\[60†L775-L781\]](#) .
- Using `response_model` helps keep the API contract explicit and secure (e.g. hiding secret fields).

- *Recommended:* Discussion (maybe illustrate with example JSON).

- **How do you handle default values and optional fields in Pydantic models?**

- *Difficulty:* Medium.

- *Key Points:*

- In Pydantic, fields with a default value (e.g. `age: int = 18`) are optional in the incoming data. You can also use `Optional[type]` or `type | None` .
- If a field has no default, it's required.
- FastAPI will not include fields with default values in request validation if the client omits them.
- For responses, use `response_model_exclude_unset=True` to omit fields left at default [\[60†L819-L824\]](#) . This avoids cluttering the JSON with defaults.
- Pydantic's `Field()` can supply default values or metadata (descriptions, examples).

- *Recommended:* Discussion with code snippet.

- **Describe major differences between Pydantic v1 and v2 that affect FastAPI.**

- *Difficulty:* Expert.

- *Key Points:*

- Pydantic v2 introduces some API changes: notably, `@validator` is replaced by `@field_validator` [\[58†L54-L59\]](#) . Many internals were rewritten for speed.

- FastAPI 0.119 introduced a temporary compatibility mode (`pydantic.v1` submodule) but this was removed by FastAPI 0.128 [\[46†L339-L347\]](#) . New apps should use Pydantic v2 ($\geq 2.7.0$) in Python 3.7+ [\[46†L273-L282\]](#) .
- Mixing v1 and v2 models is not supported (cannot nest a v2 model inside a v1 model and vice versa) [\[46†L379-L388\]](#) .
- Migration tools like `bump-pydantic` exist to update `@validator` to `@field_validator` and other syntax [\[46†L299-L307\]](#) .
- FastAPI's support for Pydantic v2 is mature, but some configuration options changed. Consult the official migration guide [\[46†L290-L294\]](#) .

• *Recommended:* Discussion (expect knowledge of `@field_validator` and version constraints).

• How can you use Pydantic's `Field()` to add metadata (e.g. descriptions, examples) and constraints?

• *Difficulty:* Medium.

• *Key Points:*

- Use `Field(...)` in model definitions to add metadata: e.g. `age: int = Field(default=0, description="Age in years", ge=0)` adds a description and enforces `age >= 0` .
- FastAPI will include these in the OpenAPI schema (showing descriptions, examples if provided).
- Other parameters: `example=`, `min_length`, `max_length`, `regex`, etc.
- Validators can also be expressed via `Field` (e.g. `gt`, `lt` for numeric bounds).

• *Recommended:* Explain with code snippet.

• When would you use a Pydantic `@root_validator` , and how does it work?

• *Difficulty:* Advanced.

• *Key Points:*

- `@root_validator` (v1) or `@model_validator(mode="before"/"after")` (v2) allows validation involving multiple fields together.
- It receives the entire data dict (v1) or model (v2) for cross-field checks.
- Example use case: ensure that `start_date < end_date` , or that at least one of two fields is provided.
- Remember to raise `ValueError` on failure.

• *Recommended:* Discussion (no code needed).

Dependency Injection

• Explain FastAPI's dependency injection system. What is a "dependency"?

• *Difficulty:* Advanced.

• *Key Points:*

- A dependency is any callable (function, class) that can provide a parameter to a path operation. Use `Depends()` in your function signature to declare it.

- FastAPI automatically “injects” these by calling the function and passing its return value to the path operation.
 - Dependencies can be anything: database session providers, security checks, configuration loaders.
 - FastAPI’s DI is based on function signature and type hints; it is “very powerful but intuitive” [31†L253-L261] .
 - Even classes (with `__call__`) or `typing.Annotated` parameters work.
- *Recommended:* Whiteboard to show an example (e.g. DB session).

• **What are sub-dependencies (dependencies of dependencies)? Give an example.**

- *Difficulty:* Advanced.
- *Key Points:*
 - Dependencies can depend on other dependencies. FastAPI resolves this dependency graph automatically [55†L639-L645] .
 - For example, an `auth_required` dependency might depend on a `get_current_user` dependency.
 - You simply declare `Depends(other_dep)` inside a dependency function. FastAPI will call each in the correct order.
 - This allows composing logic (e.g. user retrieval, then permission check).
 - Official docs call this a “hierarchical tree of dependencies” [55†L639-L645] , solved automatically.

- *Recommended:* Draw dependency graph (diagram or verbal).

• **How do you use `Depends` with classes and with `yield` ?**

- *Difficulty:* Advanced.
- *Key Points:*
 - You can define a class with an `__init__` and `__call__` and use it as a dependency by specifying `Depends(YourClass())` . FastAPI will instantiate and call it.
 - Using `yield` inside a dependency function lets you write cleanup code. For example, a database session dependency can `yield session` and then `session.close()` after the request is done.
 - This is useful for resources: e.g.

```
def get_db(): db = Session(); try: yield db; finally: db.close()
```

.
 - Dependencies with `yield` are called “context managers” – cleanup runs even if an exception occurs.

- *Recommended:* Whiteboard code example for `yield` usage.

• **What are global (app-level) dependencies, and how do they differ from router-level or endpoint-level dependencies?**

- *Difficulty:* Advanced.

- *Key Points:*
 - Global dependencies are specified with `app.dependency_overrides` or router attributes like `dependencies=[Depends(...)]` on the FastAPI app or APIRouter.
 - These run for *all* (or a set of) path operations.
 - Router-level dependencies apply to all endpoints in that router.
 - Endpoint-level dependencies (in the decorator) apply only to that endpoint.
 - Global dependencies are useful for things like requiring an API key for every endpoint. They are resolved before per-endpoint dependencies.
 - They can add overhead if heavy, so use judiciously.

• *Recommended:* Discussion (no coding needed).

• How do you override dependencies (for example, in testing)?

• *Difficulty:* Medium.

- *Key Points:*
 - FastAPI allows overriding dependencies by manipulating `app.dependency_overrides`. Example: `app.dependency_overrides[get_db] = override_get_db`.
 - This is often done in tests to provide a mock or in-memory database session.
 - When a path operation depends on `get_db`, FastAPI will use the overridden function instead.
 - Useful for injecting fake users or bypassing authentication in tests.

• *Recommended:* Brief explanation (could show code if needed).

• Why is FastAPI's DI system described as “powerful yet intuitive”? [31†L253-L261]

• *Difficulty:* Medium.

- *Key Points:*
 - It requires minimal boilerplate: dependencies are declared via function parameters using `Depends`.
 - It automatically resolves complex graphs of dependencies [31†L253-L261] [55†L639-L645] without manual wiring.
 - It enables clear separation of concerns (handlers remain small, while logic like auth or DB access is in dependencies).
 - Official docs praise its power and simplicity [31†L253-L261].

• *Recommended:* Quoted discussion (cite [31]).

Async Programming and Concurrency

• What is the difference between defining an endpoint with `async def` vs `def` in FastAPI?

• *Difficulty:* Medium.

- *Key Points:*
 - `async def` endpoints are run on the event loop; they can `await` asynchronous I/O (e.g. async DB drivers or HTTP calls).
 - Regular `def` endpoints are executed in a threadpool by Starlette, allowing use of synchronous libraries.

- FastAPI allows mixing both styles freely [24†L328-L332] . Internally, it “does the right thing” with each.
 - Use `async def` when calling async libraries; use `def` when using synchronous blocking code to avoid confusion.
- *Recommended:* Discussion (maybe show an example endpoint signature).

• How does FastAPI handle concurrency? What role do event loops and threads play?

- *Difficulty:* Advanced.
- *Key Points:*
 - FastAPI (via Uvicorn) uses Python’s `asyncio` event loop by default (ASGI server). Multiple requests are handled concurrently in the loop.
 - CPU-bound code will block the event loop, so either avoid it or run in thread/process.
 - If you use sync I/O libraries inside an `async def` , those calls will block unless you offload them.
 - Uvicorn’s `--workers` option can spawn multiple processes to utilize multiple CPU cores [49†L72-L76] [22†L61-L69] .
 - Alternatively, one process with many threads (via sync endpoints) can handle concurrency up to GIL limits.
 - Best practice: use async libraries (DB, HTTP) to maximize I/O concurrency, and use multiple workers for CPU-bound tasks [49†L72-L76] .
- *Recommended:* Whiteboard (show threads vs event loop).

• What are common pitfalls of async endpoints, and how do you avoid them?

- *Difficulty:* Advanced.
- *Key Points:*
 - Blocking calls: If you use a standard (sync) library in an `async def` , it will block the loop. Solution: use `run_in_threadpool` or use an async alternative.
 - Race conditions: Accessing shared resources (like global dicts) from async code needs locks if mutated.
 - GIL: even with asyncio, pure-Python CPU-heavy tasks cannot run truly parallel; offload to processes if needed.
 - Forgetting `await` on coroutines causes subtle bugs.
 - Deadlocks: not common in typical FastAPI use, but await cycles (rare).
- *Recommended:* Discussion (point out examples, no code needed).

• How do you manage background tasks or long-running jobs in FastAPI?

- *Difficulty:* Advanced.
- *Key Points:*
 - Use FastAPI’s `BackgroundTasks` (imported from `fastapi`) by adding a parameter to the endpoint (e.g. `background_tasks: BackgroundTasks`) and calling

`background_tasks.add_task(func, args...)` . These tasks run *after* the response is returned.

- `BackgroundTasks` are in-process and tied to the request lifecycle; they are suitable for short or medium tasks (e.g. sending an email) but will be lost if the process crashes.
- For truly long or persistent tasks, use an external task queue (Celery, RQ, etc.).
- `asyncio.create_task(...)` is also available, but doesn't wait for the result and can be forgotten on process shutdown.
- Official docs list background tasks as a feature of Starlette [\[6†L453-L461\]](#) .

• *Recommended:* Explain differences (maybe code snippet using `background_tasks`).

• **Explain the use of `--workers` with Uvicorn/FastAPI. When do you use multiple workers vs multiple containers?**

• *Difficulty:* Advanced.

• *Key Points:*

- The CLI option `--workers N` (or `workers` in Gunicorn) tells Uvicorn to spawn N separate worker processes [\[49†L72-L76\]](#) . This uses multiple CPU cores in one machine.
- Example: `fastapi run main:app --workers 4` starts 4 processes [\[49†L72-L76\]](#) .
- With modern Docker orchestration (Kubernetes), it's often preferred to run one worker per container/pod and scale horizontally. Running multiple workers per container is simpler for a single-node setup.
- FastAPI docs note that since Uvicorn now supports `--workers` , there's generally no need to use Gunicorn [\[22†L61-L69\]](#) .

• *Recommended:* Discussion (mention pros/cons, cite [\[22\]](#) and [\[49\]](#)).

• **How do you run blocking I/O calls (e.g. sync DB queries) in an async FastAPI endpoint?**

• *Difficulty:* Advanced.

• *Key Points:*

- Use Starlette's `run_in_threadpool()` function or Python's `asyncio.to_thread()` to run blocking calls without blocking the event loop.
- Example: `result = await run_in_threadpool(blocking_function, args)` .
- Alternatively, define the endpoint as `def` (sync) so Starlette runs it in a threadpool automatically.
- For databases, prefer using async DB drivers (SQLModel async engine, or libraries like `encode/databases`).

• *Recommended:* Explanation (code snippet optional).

Performance and Benchmarking

• **How can you benchmark a FastAPI application? What metrics do you collect?**

• *Difficulty:* Medium.

• *Key Points:*

- Common approaches: use load testing tools like `wrk` , `locust` , or `ab` to measure requests per second and latency under load.

- Metrics to monitor include throughput (RPS), average and 95th percentile latency, error rate, CPU and memory usage.
 - FastAPI itself reports to TechEmpower as one of the fastest frameworks [\[34†L242-L245\]](#) – you can compare against that baseline.
 - Profiling tools (cProfile, PyInstrument) can measure time spent in functions.
 - Don't forget to test with realistic data and include database or external calls if they are part of your app.
- *Recommended:* Discussion (mention specific tools, cite [\[34\]](#) for performance).

• What contributes to FastAPI's high performance? Are there any trade-offs?

- *Difficulty:* Advanced.
- *Key Points:*
 - FastAPI's performance comes from Starlette (async framework) and Uvicorn (ASGI server) which are highly optimized, plus Python 3's improvements.
 - Pydantic validation is efficient (written in C speeds, especially in v2), so data parsing is fast.
 - Automatic docs generation happens at startup, not per request, so no runtime overhead [\[34†L288-L296\]](#) .
 - It's "one of the fastest" as shown by benchmarks [\[34†L242-L245\]](#) , behind raw Uvicorn/Starlette only.
 - Trade-offs: startup time is slightly higher due to introspection, and features like detailed validation add some cost compared to a barebones framework. But documentation points out this overhead is negligible (docs generated on startup) [\[34†L288-L296\]](#) .
- *Recommended:* Discussion (cite benchmarks, [\[34\]](#) and mention docs).

• How do you optimize JSON serialization/deserialization performance?

- *Difficulty:* Advanced.
- *Key Points:*
 - By default, FastAPI (with Pydantic) uses standard `json` library under the hood. For many cases, this is already very fast. TechEmpower notes that using Pydantic models (and default JSON) yields maximum performance [\[66†L320-L327\]](#) .
 - One can use `ORJSONResponse` or `UJSONResponse` by setting `response_class` , but note the FastAPI docs advise that for maximum speed you might *not* override the default [\[66†L320-L327\]](#) .
 - If your API handles very large data payloads, ensure to use streaming responses (see next question) to avoid building huge in-memory strings.
 - Minimize large, nested structures if possible; use generators or pagination.
- *Recommended:* Discussion (cite [\[66\]](#) which says default is fastest).

• What are strategies for caching to improve FastAPI performance?

- *Difficulty:* Advanced.

- *Key Points:*

- **In-memory caching:** Use Python's `functools.lru_cache` on pure functions (if stateful endpoints allow). Not safe across multiple processes.
- **External caches:** Integrate Redis or Memcached to cache expensive query results or computation. E.g. cache GET responses or heavy DB queries.
- **HTTP caching:** Use appropriate response headers (ETag, Cache-Control) or client-side caches.
- FastAPI doesn't have built-in cache middleware, so this is typically done at the application or infrastructure level.
- Example: memoize results of a slow function with `cachetools` or use `fastapi-cache` extension.

- *Recommended:* Conceptual discussion.

- **How can you profile or analyze FastAPI performance issues in code?**

- *Difficulty:* Advanced.

- *Key Points:*

- Use Python profilers (`cProfile`, `PyInstrument`, `line_profiler`) on endpoints to find slow functions.
- Use metrics libraries (e.g. `prometheus-client`) to record request durations and database timings.
- Use logging or APM tools (New Relic, Datadog) for distributed tracing.
- Identify heavy Pydantic operations: `.dict()` or `.json()` calls.
- Benchmark specific code paths with simple timers.

- *Recommended:* Explain steps/tools.

- **What does TechEmpower benchmarking say about FastAPI performance?**

- *Difficulty:* Medium.

- *Key Points:*

- Official FastAPI docs cite TechEmpower: FastAPI (with Uvicorn) is *among the fastest* Python frameworks [34†L242-L245] . It's only behind Starlette and Uvicorn themselves.
- This benchmark is in a simple "JSON serialization" test.
- The key takeaway is that FastAPI adds very little overhead over Starlette while providing validation and docs.
- (Cite: "TechEmpower benchmarks show FastAPI as one of the fastest frameworks available" [34†L242-L245]).

- *Recommended:* Discussion (refer to [34]).

- **How can you efficiently handle large file uploads or downloads?**

- *Difficulty:* Medium.

- *Key Points:*

- For uploads: use `UploadFile` (returns `SpooledTemporaryFile`) instead of raw bytes to avoid loading the entire file in memory. This allows async streaming.

- For downloads/streaming: use `StreamingResponse` and yield data chunks. Do **not** return a huge string.
- Streaming example: return `StreamingResponse(iterator, media_type="application/octet-stream")`. See FastAPI docs on streaming [\[53†L557-L560\]](#).
- Always set appropriate `media_type` or file attachments (`FileResponse` helper).
- *Recommended:* Discussion (mention [\[53\]](#) as reference to streaming data).

Security and Authentication/Authorization

• What built-in security utilities does FastAPI provide? [\[4†L394-L400\]](#)

- *Difficulty:* Medium.
- *Key Points:*
 - FastAPI includes helpers for HTTP Basic Auth, OAuth2 (with password or other flows), API key (header/query/cookie), HTTP Bearer tokens (e.g. JWT) [\[4†L394-L400\]](#).
 - These are provided as dependencies (`fastapi.security` classes) that can be included in endpoints.
 - You can also use Starlette's session middleware for cookie-based sessions.
 - For OAuth2 with JWT, FastAPI has utilities like `OAuth2PasswordBearer` and example code in docs.
- *Recommended:* Discussion (cite [\[4\]](#) for supported schemes).

• Explain how you would implement OAuth2 password flow with JWT tokens in FastAPI.

- *Difficulty:* Advanced.
- *Key Points:*
 - Use `OAuth2PasswordBearer(tokenUrl="token")` as a dependency to get the token from the Authorization header.
 - Create an endpoint `/token` that uses `OAuth2PasswordRequestForm` to receive username/password, verify credentials, and return a signed JWT.
 - Use a library like `PyJWT` or `python-jose` to encode/decode JWTs.
 - Protect other endpoints by a dependency that validates the JWT and returns the current user (e.g. `get_current_user`).
 - Define scopes if needed (the `scopes` parameter in `OAuth2PasswordBearer`).
 - FastAPI documentation and features support this flow natively [\[4†L394-L400\]](#).
- *Recommended:* Discussion (no need to write full code).

• How would you add HTTP Basic or API Key authentication to an endpoint?

- *Difficulty:* Medium.
- *Key Points:*
 - For HTTP Basic: use `from fastapi.security import HTTPBasic, HTTPBasicCredentials`. Declare a parameter like `credentials: HTTPBasicCredentials = Depends(HTTPBasic())`. Then check `credentials.username/password`.

- For API key: use `APIKeyHeader`, `APIKeyQuery`, or `APIKeyCookie`. For example: `api_key: str = Depends(APIKeyHeader(name="X-API-Key"))`. Validate the key against your store.
- If invalid, raise `HTTPException(401)`.
- These tools ensure proper reading of headers and forming of security scheme in OpenAPI docs.

• *Recommended:* Write small example or explain.

• How can you enforce role-based access control (RBAC) or permissions in FastAPI?

• *Difficulty:* Advanced.

• *Key Points:*

- Typically by adding logic in a dependency (e.g. `get_current_user`) to check `user.role` or scopes. Raise `HTTPException(403)` if not allowed.
- You might create a dependency factory: `def require_role(role): ...` that returns a Depends-able function.
- For OAuth2 scopes: use the `scopes` parameter in `OAuth2PasswordBearer` and validate the token contains the scope.
- Document roles/scopes in the OpenAPI schema by passing `scopes` to the dependency.

• *Recommended:* Discussion or whiteboard (draw flow of token->roles).

• What is CORS and how do you configure it in FastAPI? [69†L25-L34]

• *Difficulty:* Medium.

• *Key Points:*

- CORS (Cross-Origin Resource Sharing) controls which web origins can access your API from a browser.
- In FastAPI, add the built-in `CORSMiddleware` (from `fastapi.middleware.cors`) to the app.
- Configure `allow_origins`, `allow_methods`, `allow_headers`, etc. Example:

```
python
from fastapi.middleware.cors import CORSMiddleware
app.add_middleware(
    CORSMiddleware,
    allow_origins=["https://example.com", "http://localhost:3000"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

 [69†L25-L34] .

- Without this, browser clients may be blocked when calling your API from a different origin.

• *Recommended:* Brief code snippet (cite [69]).

• How do you protect against common web vulnerabilities (e.g., SQL injection, XSS, CSRF) in FastAPI?

- *Difficulty:* Medium.
- *Key Points:*
 - **SQL Injection:** Use parameterized queries or ORM (SQLAlchemy) which auto-escapes input. Never concatenate SQL strings from user input.
 - **XSS:** FastAPI returns JSON by default (not rendering HTML), so direct XSS is rare. If you do return HTML, ensure you escape content or use templating auto-escaping (Jinja2).
 - **CSRF:** If using cookie-based auth, implement CSRF tokens (FastAPI itself doesn't have built-in, but you can use Starlette's `SessionMiddleware` and CSRF libraries). For token-based APIs (JWT), CSRF is less an issue since tokens are sent in headers.
 - **HTTPS:** Always run FastAPI behind TLS/SSL (either via reverse proxy or using `--ssl` options). Use `--proxy-headers` if behind a proxy to get correct scheme `【77†L7-L14】`.
 - **Input validation:** Rely on Pydantic to reject invalid data, reducing injection risk.
 - **CORS:** Configure as above to limit frontend access.
- *Recommended:* Discussion (conceptual).

• How would you implement rate limiting or request throttling in FastAPI?

- *Difficulty:* Advanced.
- *Key Points:*
 - FastAPI has no built-in rate limiter; use a third-party middleware or proxy.
 - Options include using a reverse proxy (Nginx, API Gateway) with rate-limiting rules, or integrating a library like `slowapi` (based on `limits`).
 - Another approach: track requests per client in Redis and block when threshold is reached.
 - In path operations, you could use a dependency to check a counter in memory/DB and raise an exception if exceeded.
- *Recommended:* Conceptual (since it's more design than code).

Testing and Debugging

• How do you test FastAPI endpoints? Demonstrate using `TestClient`. `【73†L43-L51】`

- *Difficulty:* Medium.
- *Key Points:*
 - Import `TestClient` from `fastapi.testclient` (or Starlette).
 - Create it with your app: `client = TestClient(app)`.
 - Use it like an HTTP client: `response = client.get("/path")`.
 - Check `response.status_code`, `response.json()`, etc.
 - Example from docs:

```
python
from fastapi.testclient import TestClient
app = FastAPI()
client = TestClient(app)
def test_read_main():
    response = client.get("/")
    assert response.status_code == 200 【73†L43-L51】
```
 - Supports JSON, query params, headers just like real requests.
- *Recommended:* Live coding (write a simple test).

• How can you test async endpoints or WebSockets?

• *Difficulty:* Advanced.

• *Key Points:*

- For async path ops, `TestClient` handles them the same as sync, because it runs the event loop under the hood (HTTPX supports asyncio).
- For async testing frameworks, you can use `httpx.AsyncClient` with `pytest-asyncio`.
- For WebSockets, Starlette provides a test client that supports `client.websocket_connect("/ws")` and `.receive_text()`, `.send_text()`.
- FastAPI docs have examples for testing WebSockets (using `client.websocket_send` and `client.websocket_receive`) [85†L848-L855] .

• *Recommended:* Describe approach; no full code needed.

• How do you override dependencies in tests?

• *Difficulty:* Advanced.

• *Key Points:*

- FastAPI allows you to set `app.dependency_overrides[dependency_function] = override_function`.
- For example, you can override `get_current_user` to always return a test user.
- The override is applied only while the test client is running.
- Example:

```
app.dependency_overrides[get_db] = lambda: test_db_session
```

- Remember to clear `dependency_overrides` after tests if needed.

• *Recommended:* Explain with a code snippet.

• How do you handle database testing (e.g., using SQLite in-memory)?

• *Difficulty:* Advanced.

• *Key Points:*

- Use a separate database for testing, often SQLite in-memory or a dedicated test DB.
- Create the tables at test startup and tear them down after.
- Override the normal `get_db` dependency to return a session connected to the test database.
- Use fixtures (pytest) to set up and tear down DB state.
- FastAPI docs show how to test with databases by overriding session dependencies.

• *Recommended:* Discussion or pytest fixture example.

• What are strategies to debug FastAPI applications?

• *Difficulty:* Medium.

- **Key Points:**
 - Use Python's `logging` (configure log level, format) to output debug info. Uvicorn logs can be set with `--log-level debug`.
 - Use `import pdb; pdb.set_trace()` or VS Code/IDE breakpoints.
 - Enable `--reload` in development to auto-restart on code changes.
 - Use exception tracebacks (FastAPI shows detailed errors in debug mode).
 - The interactive docs themselves help verify the API without writing clients.
 - For tracing: use tools like Sentry or OpenTelemetry.
- **Recommended:** Discussion.

Deployment and DevOps

- **How do you containerize a FastAPI application with Docker? What are best practices?**
 - **Difficulty:** Medium.
 - **Key Points:**
 - Use a Python base image, install dependencies, copy code. Use multi-stage builds to minimize image size.
 - Always use `exec` form in `CMD` (e.g. `CMD ["fastapi", "run", "app/main.py", "--port", "80"]`) to allow signal propagation [\[21†L579-L588\]](#) [\[21†L593-L601\]](#).
 - Avoid installing dev dependencies in the final image. Pin dependencies for reproducibility.
 - If static files/templates are needed, copy them into the image or use a volume.
 - Configure environment variables for secrets/config (don't bake secrets into image).
 - Expose the port (e.g. 80) and use `HEALTHCHECK` instructions if desired.
 - **Recommended:** Brief code snippet of Dockerfile (no `exec` vs shell explanation).
- **What options exist for deploying FastAPI (Uvicorn vs Gunicorn vs Hypercorn)?**
 - **Difficulty:** Medium.
 - **Key Points:**
 - The simplest is to use Uvicorn directly (it's the recommended ASGI server for FastAPI). Example: `uvicorn app.main:app --host 0.0.0.0 --port 80`.
 - Historically people used Gunicorn+Uvicorn workers, but as of recent FastAPI docs note that Uvicorn *itself* supports `--workers` (multiprocessing) so Gunicorn is not strictly needed [\[22†L61-L69\]](#).
 - Hypercorn is another ASGI server alternative (supports HTTP/2/3, asyncio).
 - In Docker/Kubernetes, either run `uvicorn` in one process per container, or a process manager like Gunicorn can be used (though Uvicorn's own worker mode often suffices).
 - Always consider how to handle signals for graceful shutdown (`uvicorn` and Gunicorn handle this differently).
 - **Recommended:** Discussion (cite [\[22\]](#) on workers).
- **Explain how you would set up a 90-minute interview (time breakdown).**

(Note: This was part of table summary below, not a question to ask the candidate.)
- **How do you use HTTPS/TLS and proxies with FastAPI?**

- *Difficulty:* Medium.

- *Key Points:*

- In production, FastAPI is typically run behind a reverse proxy (Nginx, Traefik, AWS ELB) that terminates TLS.
- To make FastAPI aware of the original protocol and host, use the `--proxy-headers` flag (or set `UVICORN_APP="app:app"` with `uvicorn.run(..., proxy_headers=True)` [77†L7-L14]).
- Uvicorn can also serve TLS directly with `--ssl-keyfile` / `--ssl-certfile` (less common).
- Ensure certificates are properly managed (Let's Encrypt, cloud provider).

- *Recommended:* Discussion (cite [77]).

- **Describe a basic CI/CD pipeline for a FastAPI project.**

- *Difficulty:* Medium.

- *Key Points:*

- **CI (Continuous Integration):** On each commit or PR, run linting (flake8/mypy) and run the test suite (`pytest`).
- **Build:** If all tests pass, build a Docker image (with version tag or SHA tag). Push to a registry.
- **CD (Continuous Deployment):** Deploy the container to staging/production (e.g. update Kubernetes deployment or trigger a serverless deploy).
- Include migration steps (e.g. run Alembic migrations at startup or in pipeline).
- Use environment variables or secrets for configuration (12-factor app).

- *Recommended:* List steps (no coding needed).

- **How can you achieve zero-downtime deployments or graceful restarts?**

- *Difficulty:* Advanced.

- *Key Points:*

- Use multiple replicas (Kubernetes rolling updates or Docker swarm) so new pods start before old ones shut down.
- Uvicorn (and Gunicorn) will stop accepting new requests on shutdown and finish existing ones (graceful shutdown).
- The **exec** CMD form allows signals (SIGTERM) to reach Uvicorn to trigger shutdown events [21†L579-L588] .
- Use health checks so orchestrators only route traffic to ready instances.

- *Recommended:* Discussion (mention [21] on exec form).

- **How do you integrate logging and monitoring for production FastAPI (e.g. Prometheus, Grafana)?**

- *Difficulty:* Advanced.

- *Key Points:*

- FastAPI/Starlette can expose metrics via middleware or dependencies. For example, use `prometheus_client` library and expose an endpoint like `/metrics` .

- Uvicorn access logs and application logs can be shipped to a log aggregator (ELK, Graylog).
- Use structured logging (JSON, include request IDs).
- For error monitoring, integrate Sentry or similar.
- Grafana/Prometheus can scrape metrics (response times, error rates) to visualize.
- *Recommended:* Conceptual (no code needed).

Database Integration and ORMs

- **How do you typically integrate a SQL database (e.g. PostgreSQL) with FastAPI?** [82†L898-L902]

• *Difficulty:* Medium.

• *Key Points:*

- Common pattern: use SQLAlchemy (or SQLModel) for ORM. Create an `Engine` and a `SessionLocal`.
- Define a dependency that yields a database session per request. For example (SQLModel):

```
def get_session():
    with Session(engine) as session:
        yield session
```

FastAPI will call `session.close()` after the request [82†L898-L902] .

- Use Pydantic (or SQLModel) models to parse input.
- Apply transactions (`session.commit()` / `rollback()`) as needed within the endpoint.
- The FastAPI docs provide a tutorial on SQL (Relational) databases showing exactly this pattern [82†L898-L902] .

• *Recommended:* Discussion with code example (cite [82]).

- **What are the pros and cons of async ORMs vs synchronous ORMs in FastAPI?**

• *Difficulty:* Advanced.

• *Key Points:*

- **Sync ORMs (SQLAlchemy ORM 1.x):** Mature, feature-rich, but blocking. Must use them carefully in async context (offload or use threadpool).
- **SQLModel (sync by default):** Convenient but not async.
- **Async ORMs (SQLAlchemy 1.4+ async, Tortoise-ORM, GINO):** Non-blocking I/O (ideal for async endpoints) but sometimes less mature or feature-complete.
- Async ORMs require an `async` context and often a different syntax (e.g. `async with session.begin()`).
- If using async DB access, your endpoints should be `async def` to await the calls.
- Choose based on workload: if high concurrency is needed and DB driver supports async (e.g. asyncpg), async ORM can scale better.

• *Recommended:* Discussion.

- **How do you manage DB sessions and transactions in FastAPI?**

- *Difficulty:* Advanced.

- *Key Points:*

- Using the dependency with `yield` ensures a session per request. Typically you `session.commit()` when data changes. If an exception occurs, do `session.rollback()`.
- A common pattern:

```
def get_session():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

- Use `@app.on_event("startup")` to run migrations or create tables (one-time DB setup) `[82†L910-L918]` `[94†L324-L332]`.
- For multi-step operations, you can use `session.begin()` context managers for atomic transactions.
- In async ORMs, use `async with session.begin(): ...`.

- *Recommended:* Conceptual discussion.

• How do you perform database migrations in a FastAPI project?

- *Difficulty:* Medium.

- *Key Points:*

- Typically use Alembic for SQLAlchemy/SQLModel. Define migration scripts for schema changes.
- Run `alembic upgrade head` as part of deployment or startup event.
- For SQLModel, there's also `sqlmodel migrate` based on Alembic.
- For NoSQL, migrations are manual (scripts) or use ODM-specific tools (e.g. MongoEngine migrations).
- In CI/CD, migrations can run automatically after deploying a new image but before cutting traffic.

- *Recommended:* Discussion (mention Alembic or similar).

• Explain how to work with relationships (one-to-many, many-to-many) using SQLAlchemy in FastAPI.

- *Difficulty:* Advanced.

- *Key Points:*

- Define relationships in your ORM models (e.g. `relationship()` in SQLAlchemy).
- Use `pydantic models` (e.g. `response_model`) carefully to avoid recursion: you might exclude backrefs or use nested models.
- Example: a `User` has many `Item`s. Define `items = relationship("Item", back_populates="owner")` and `owner = relationship("User")` in `Item`.

- Use joins or lazy loading appropriately.
- Be aware of N+1 query problems: fetch related data with `.options(joinedload(...))` if needed.
- *Recommended:* Discussion or code (no need for references).

• **How would you use a NoSQL database (e.g., MongoDB) or a key-value store with FastAPI?**

- *Difficulty:* Medium.
- *Key Points:*
 - You can use any DB by writing your own dependency. For MongoDB, use an async driver like Motor or an ODM like Beanie (Pydantic-based).
 - Example (Motor):

```
from motor.motor_asyncio import AsyncIOMotorClient
client = AsyncIOMotorClient(MONGODB_URL)
db = client.test
async def get_mongo():
    yield db
```

- For key-value (Redis), use `redis-py` or `aioredis`. Connect in startup, pass through dependency, use `await redis.get(...)`.
- In all cases, ensure connections are reused (avoid reconnecting on every request). Close them on shutdown event.
- *Recommended:* Discussion.

• **What is the significance of `Base.metadata.create_all(engine)` and where would you put it?**

- *Difficulty:* Medium.
- *Key Points:*
 - `create_all()` creates database tables for all models not already present. It's useful for quick startups or SQLite demos.
 - In production, often migrations are used instead, but you might call it in a `startup` event for simple setups.
 - In the FastAPI tutorial, it's called in `@app.on_event("startup")` before serving requests [82†L904-L912] .
 - Don't put it in the middle of request handling – it should run once at startup.
- *Recommended:* Discussion (cite [82] where startup event creates tables).

WebSockets and Background Tasks

- **How do you implement a WebSocket endpoint in FastAPI? Provide a brief example.** [85†L815-L823]
- *Difficulty:* Advanced.

- *Key Points:*

- Use `@app.websocket("/ws/{client_id}")` to declare a WebSocket route. The handler receives a `WebSocket` object.

- Example from docs:

```
python
@app.websocket("/ws/{client_id}")
async def websocket_endpoint(websocket: WebSocket, client_id: int):
    await manager.connect(websocket)
    try:
        while True:
            data = await websocket.receive_text()
            await manager.send_personal_message(f"You wrote: {data}",
            websocket)
            # broadcast to others...
    except WebSocketDisconnect:
        manager.disconnect(websocket) 【85†L815-L823】
```

- Remember to call `await websocket.accept()` (usually in the connect helper) and handle `WebSocketDisconnect`.
- The example uses a `ConnectionManager` to track clients and broadcast messages 【85†L815-L823】.

- *Recommended:* Code review (go over example, see [85]).

- **What are the limitations of using in-process WebSocket managers (like the one above)?** 【85†L848-L855】

- *Difficulty:* Advanced.

- *Key Points:*

- The provided example stores connections in a Python list in memory. This works *only* while that process is running and with a single process.
- In a multi-worker environment, each worker would have its own separate list (no shared state).
- For a robust solution, use an external system: e.g. [Broadcaster](#) with Redis/Postgres for Pub/Sub, or run a separate WebSocket service (e.g. Socket.IO, or MQTT).
- The docs explicitly warn that the memory-based approach “will only work with a single process” and suggest using external tools 【85†L848-L855】.

- *Recommended:* Discussion (cite [85]).

- **How do you use FastAPI's `BackgroundTasks` ? Give an example.** 【88†L1-L4】

- *Difficulty:* Medium.

- *Key Points:*

- Import `BackgroundTasks` from FastAPI and include it as a parameter in a path operation. FastAPI will inject it.
- Call `background_tasks.add_task(func, *args, **kwargs)` to schedule `func` to run after the response is returned.

- Example from docs:

```
python
@app.post("/notify/{email}")
def send_notification(email: str, background_tasks: BackgroundTasks):
    background_tasks.add_task(write_notification, email,
message="Hello")
    return {"message": "Notification scheduled"} # immediate response
```

- This will run `write_notification(email, "Hello")` in the background (in the same process). Good for tasks like sending emails, without delaying the API response.
- For long-running or persistent tasks, use an external queue (Celery, RQ) instead.

• *Recommended:* Code writing (simple example).

• **Explain the difference between using `BackgroundTasks` and an external task queue (e.g. Celery) in a FastAPI app.**

• *Difficulty:* Advanced.

• *Key Points:*

- `BackgroundTasks` runs tasks **in the same process** after returning the response. It's simple but not suitable for very long or critical tasks (if the process crashes, task is lost).
- External queues (Celery, RQ) run in separate worker processes or machines. They persist tasks (often backed by Redis/RabbitMQ). They handle retries, failures, and can scale independently.
- Use `BackgroundTasks` for quick, non-critical jobs. Use Celery for heavier tasks that need reliability or need to run outside the web server's lifecycle.

• *Recommended:* Discussion.

• **How do you implement server-sent events (SSE) in FastAPI?**

• *Difficulty:* Medium.

• *Key Points:*

- SSE can be implemented with `StreamingResponse`. Set `media_type="text/event-stream"` and yield data in SSE format (`"data: ...\n\n"`).
- FastAPI doesn't have a built-in SSE class, but any async generator of strings/bytes can stream.
- Example:

```
async def event_generator():
    while True:
        yield f"data: {random_message()}\n\n"
        await asyncio.sleep(1)
@app.get("/stream")
async def stream():
    return StreamingResponse(event_generator(), media_type="text/
event-stream")
```

- Clients can listen for messages via the EventSource API in JS.
- *Recommended:* Explain concept (no full code needed).

API Design and Versioning

• What strategies can you use to version a FastAPI API?

• *Difficulty:* Medium.

• *Key Points:*

- Common approach: include version in the URL path (e.g. `/v1/items`, `/v2/items`). Create separate API Routers for each version and include them with `app.include_router(router_v1, prefix="/v1")`.
- Header versioning: read a custom header (e.g. `Accept: application/vnd.myapi.v1+json`) via a dependency, and route accordingly. Less common and more complex to document.
- Domain versioning: put version in subdomain (`v1.api.example.com`).
- FastAPI has no built-in versioning, so it's a design choice. Provide clear documentation for each version.

• *Recommended:* Discussion (possibly diagram different paths).

• How do you handle API deprecation and backward compatibility?

• *Difficulty:* Advanced.

• *Key Points:*

- Keep older endpoints live while migrating clients. Return a deprecation notice in headers or response (e.g. a custom warning header).
- Clearly document deprecated paths and a removal timeline.
- Use versioning (new version retains old behavior if needed).
- Maintain compatibility flags in code (feature toggles).
- If using Pydantic models, you can include both new and old fields with conditional response depending on version.

• *Recommended:* Discussion.

• How would you implement pagination and filtering in a FastAPI endpoint?

• *Difficulty:* Medium.

• *Key Points:*

- Use query parameters like `limit` and `offset` (e.g. `limit: int = 100, offset: int = 0` with constraints) to paginate results. The SQL example in FastAPI docs uses `select(...).offset(offset).limit(limit)` [\[82†L917-L925\]](#).
- For cursor-based pagination, use an opaque cursor (e.g. an encoded ID or timestamp) instead of offset.
- Filtering: accept query params for filter criteria (e.g. `?category=books`). Use Pydantic query models if many filters.
- Document pagination parameters in OpenAPI (they will appear in the docs).

- *Recommended:* Explain (cite [82] snippet if needed).

• What are some REST API design best practices (HTTP methods, status codes, naming)?

- *Difficulty:* Medium.

- *Key Points:*

- Use nouns (usually plural) for resource paths (`/users` not `/getUsers`).
- Use appropriate HTTP methods: GET for reads, POST for create, PUT/PATCH for update, DELETE for delete.
- Return proper status codes: 200/201/204 for success, 404 for not found, 400 for bad requests, etc. FastAPI's `HTTPException` helps set codes.
- Consistent error format (e.g., JSON with keys like `detail`). FastAPI's default is `{"detail": "..."} .`
- Maintain statelessness, use HATEOAS (if desired) or at least self-describing links if needed.
- Version your API as discussed.

- *Recommended:* Discussion.

• How do you design meaningful error responses in FastAPI?

- *Difficulty:* Medium.

- *Key Points:*

- FastAPI's default is to return JSON with a `detail` field when using `HTTPException` . You can customize by raising `HTTPException(status_code=422, detail=[{loc, msg, ...}, ...])` to include multiple errors.
- You can also define custom exception handlers (`@app.exception_handler`) that format errors in a specific JSON schema.
- Include useful information but avoid leaking internal details. For example, include which fields failed validation.
- Align with OpenAPI's standard error schema if using `detail` .

- *Recommended:* Discussion (no code needed).

Error Handling and Logging

• How do you raise an HTTP error (e.g. 404) in FastAPI? [92†L295-L300]

- *Difficulty:* Medium.

- *Key Points:*

- Use `fastapi.HTTPException` . E.g.:

```
if item_id not in items:
    raise HTTPException(status_code=404, detail="Item not found")
```

This is shown in the docs [92†L295-L300] .

- This will automatically return a JSON response `{"detail": "..."} .` with the given status code.
- For custom headers (e.g. authentication challenges), you can pass a `headers={...}` dict.

- *Recommended:* Code writing (simple example, cite [92]).

• How do you write a custom exception handler in FastAPI?

- *Difficulty:* Advanced.

- *Key Points:*

- Use `@app.exception_handler(SomeException)` decorator. The function takes `(request, exc)` and returns a `Response` (JSON or custom).
- Example:

```
from fastapi.responses import JSONResponse

@app.exception_handler(MyCustomError)
async def unicorn_exception_handler(request, exc):
    return JSONResponse(
        status_code=418,
        content={"message": f"Oops! {exc.name} did something."},
    )
```

- You can override the default handler for `HTTPException` or `RequestValidationError` similarly.
- Don't forget to import the exceptions from FastAPI/Starlette.

- *Recommended:* Discussion (mention example form).

• What is FastAPI's default format for validation errors?

- *Difficulty:* Medium.

- *Key Points:*

- FastAPI (via Pydantic/Starlette) returns validation errors as JSON with a list of error objects. Example:

```
{
  "detail": [
    {"loc": ["body", "field"], "msg": "field required", "type":
"value_error.missing"},
    ...
  ]
}
```

- Each error has `loc` (location), `msg` (message), and `type`.
- You can customize this by setting a `RequestValidationError` handler.

- *Recommended:* Explanation (no citations needed).

• How do you configure and use Python logging in FastAPI?

- *Difficulty:* Medium.

- *Key Points:*

- FastAPI (Uvicorn) uses the standard `logging` module. You can create loggers (e.g. `logger = logging.getLogger("myapp")`) and configure handlers/formatters in code or via `logging.config.dictConfig`.
- Uvicorn's default access logger logs each request. You can adjust its level via `--log-level` or in `uvicorn.run()` parameters.
- For JSON structured logs, use libraries like `loguru` or `python-json-logger`.
- It's common to log at INFO or DEBUG for requests and ERROR for exceptions.
- You can also add middleware (like above) to log request/response details (path, time) as needed.

- *Recommended:* Discussion.

- **How can you measure request processing time in FastAPI? (Hint: middleware) [97†L294-L301]**

- *Difficulty:* Advanced.

- *Key Points:*

- Use a custom middleware to capture start time and end time around `call_next(request)`.
- The official example adds an `X-Process-Time` header:

```
@app.middleware("http")
async def add_process_time_header(request: Request, call_next):
    import time
    start_time = time.perf_counter()
    response = await call_next(request)
    process_time = time.perf_counter() - start_time
    response.headers["X-Process-Time"] = str(process_time)
    return response
```

(See docs [97†L294-L301] .)

- You can also log the time or push it to metrics.

- *Recommended:* Review code (cite [97]).

Migration and Backward Compatibility

- **How should you approach upgrading FastAPI to a new version?**

- *Difficulty:* Medium.

- *Key Points:*

- Write a comprehensive test suite for your app. FastAPI recommends this first [94†L324-L332] .
- Bump the FastAPI version and run tests. Fix any breaking changes.
- Pin FastAPI (and any other dependencies) to ensure reproducibility.
- Starlette is a dependency: do *not* pin Starlette manually, let FastAPI manage it [94†L337-L342] .

- For Pydantic, use versions $\geq 2.7.0$ and $< 3.0.0$ for compatibility [\[94†L343-L352\]](#) .
- *Recommended:* Discussion (cite [\[94\]](#)).
- **What challenges arise when upgrading Pydantic from v1 to v2 in a FastAPI project?**
- *Difficulty:* Advanced.
- *Key Points:*
 - As discussed, `@validator` vs `@field_validator` . You need to update decorators.
 - Pydantic v2 is stricter by default (some type coercions changed), which may cause validation errors. The migration guide highlights these [\[46†L290-L294\]](#) .
 - If you used FastAPI v0.119-0.127, it allowed interop with `pydantic.v1` via `fastapi.params.Body(...)` fallback [\[46†L339-L347\]](#) , but newer FastAPI dropped that.
 - Updating involves running `bump-pydantic` tool or manual changes, then thorough testing.
- *Recommended:* Explanation.

- **How do you handle backward compatibility when changing request/response schemas?**

- *Difficulty:* Medium.
- *Key Points:*
 - You can keep old fields optional and handle both old/new parameters in code. Mark fields deprecated in docs if possible.
 - Use semantic versioning for your API contract; introduce breaking changes only in major version bump.
 - FastAPI doesn't auto-handle compatibility, so it's up to API design (e.g. create v2 endpoints).
 - Pydantic's alias feature can help accept both old and new field names temporarily (`Field(alias="oldName")`).
- *Recommended:* Discussion.

- **What is the purpose of the FastAPI “app.include_router” with tags and prefixes for versioning?**

- *Difficulty:* Medium.
- *Key Points:*
 - `include_router` allows grouping routes (e.g. by version or feature). You can assign `prefix="/v1"` and `tags=["v1"]` so that all those routes share a URL segment and group in the docs.
 - This is a clean way to manage multiple API versions or logical divisions (auth, items, etc.).
 - It avoids duplicating code and makes it easy to later drop an entire version or group.
- *Recommended:* Explanation.

Real-World Scenarios / Case Studies

- **Design a URL shortener service using FastAPI.** (*Scenario*)
- *Difficulty:* Expert.

- *Key Points:*

- Endpoints: `POST /shorten` (accept original URL, return short code); `GET /{code}` (redirect to original).
- Database: a simple table mapping codes to URLs. Use a fast key-value store or SQL with indexing on code.
- Generate unique codes (e.g. base62). Handle collisions.
- Caching: frequently accessed URLs could be cached in Redis.
- Deploy multiple FastAPI instances behind a load balancer.
- Security: rate limit shorten requests to prevent abuse.
- Include a code diagram:

```
flowchart LR
    A[User] -->|POST /shorten| B[FastAPI Service]
    B --> C[(SQL or NoSQL DB)]
    B --> D[(Redis Cache)]
    B -->|return code| A
    A -->|GET /{code}| B
    B --> C
    C --> B
    B -->|HTTP 302 Redirect| A
```

This shows user interactions with the FastAPI service, database, and cache.

- *Recommended:* Discussion/whiteboard (walk through design, see diagram).

- **How would you design a real-time chat system using FastAPI and WebSockets?** (*Scenario*)

- *Difficulty:* Expert.

- *Key Points:*

- Use WebSockets (`@app.websocket("/ws")`). Manage connections with an in-memory manager or external pub/sub (e.g. Redis Channels) for horizontal scaling.
- Each chat room could be a topic; messages received are broadcast to all clients in that room.
- Persist chat messages in a database (noSQL or time-series DB) if needed.
- Handle authentication at handshake time (e.g. query param token, or a separate REST call to authorize).
- Example sequence: user connects → FastAPI verifies user, adds to `ConnectionManager` (like in docs) → user sends message → server broadcasts to others.
- For scaling: use a message broker (e.g. Redis Pub/Sub) so that different FastAPI instances can broadcast messages to all.

- *Recommended:* High-level design discussion.

- **Suppose you have a long-running data processing job. How would you trigger it from FastAPI and monitor its progress?** (*Scenario*)

- *Difficulty:* Advanced.
- *Key Points:*
 - Trigger job: likely via an endpoint that enqueues the task (e.g. push to Celery queue). Return a job ID.
 - Monitoring: provide another endpoint to check status (`GET /jobs/{id}`), which checks job state (from DB or Celery result backend).
 - Use WebSockets or Server-Sent Events to push progress updates to the client if needed in real-time.
 - Don't tie up the request: run job in background (Celery, Redis, or even `BackgroundTasks` if short).

• *Recommended:* Discussion (flow: API call → queue → worker → database → status endpoint).

• **How would you migrate an existing Flask/Django monolithic API to FastAPI?** (*Scenario*)

- *Difficulty:* Advanced.
- *Key Points:*
 - Identify endpoints: rewrite routes as FastAPI path operations, reusing data models (e.g. with Pydantic).
 - Authentication: ports to FastAPI's dependencies (e.g. convert Flask-Login to FastAPI Security).
 - Database models (SQLAlchemy, etc) can largely remain, only change session handling.
 - Test thoroughly (use migration tools or libraries like `fastapi-apispec`).
 - Performance gain expected by adopting async where possible (e.g. async DB or HTTP libraries).

• *Recommended:* Discussion (no coding, focus on strategy).

• **Code Review Exercise:** (*Scenario*)

Given a code snippet:

```
@app.get("/items")
def read_items():
    items = db.query(Item).all()
    return [item.dict() for item in items]
```

Identify potential issues and improvements.

- *Difficulty:* Expert.
- *Key Points:*
 - The function is `def`, so it runs in a thread (fine here), but using synchronous ORM in an async context should be deliberate.
 - Returning `[item.dict() for item in items]` is redundant: SQLAlchemy models aren't Pydantic. We could use a response model and let FastAPI serialize.
 - Fetching all items without pagination may be a problem for large tables. Suggest adding limit/offset.

- Better to use `.all()` is fine, but could also consider streaming or more efficient query if large data.
 - Need to close `db` session (missing context or dependency).
 - Could simply do `return items` if `items` are Pydantic models (with `response_model`), to let FastAPI handle JSON.
- *Recommended:* Code review (ask candidate to critique and suggest fixes).
- **How do you optimize a Pydantic model if you only need a subset of fields frequently?**
- *Difficulty:* Advanced.
 - *Key Points:*
 - You can use `response_model_include` in the decorator to only output certain fields. Or define a separate Pydantic model with fewer fields.
 - Use `response_model_exclude_unset` if fields have defaults.
 - Avoid `model.dict()` on large objects repeatedly; prefer specifying `__dict__` or SQLAlchemy serializer if speed critical.
 - Pydantic v2 is faster, so upgrading can help with large models.
 - *Recommended:* Discussion.

Code Review and Optimization

- **When reviewing FastAPI code, what performance or security best practices do you look for?**
 - *Difficulty:* Advanced.
 - *Key Points:*
 - Ensure proper use of async vs sync (no blocking calls in async endpoints).
 - Check for missing dependency cleanup (`yield` usage).
 - Validate that Pydantic models are used for inputs/outputs to ensure type safety.
 - Look for repeated database calls in loops (should batch queries).
 - Confirm that secrets (API keys, DB URIs) aren't hard-coded.
 - Ensure logging isn't too verbose or missing for errors.
 - Security: no broad exception catching that hides errors; proper use of `HTTPException` for access control.
 - *Recommended:* Discussion (general advice).
- **How can you reduce memory usage in FastAPI when dealing with large responses?**
- *Difficulty:* Medium.
 - *Key Points:*
 - Stream the response instead of building a giant list/dict in memory. Use `StreamingResponse` for large data or file downloads.
 - Use generators or pagination to handle big query results.
 - Limit Pydantic models to fields needed. Avoid loading entire DB objects if not necessary.
 - *Recommended:* Discussion.

- **What is the effect of using `@lru_cache` on a dependency function in FastAPI?**

- *Difficulty:* Medium.

- *Key Points:*

- Decorating a dependency function with `@lru_cache` makes it run only once per process, caching its return value. Useful for expensive lookups (e.g. loading settings from DB) that don't change per request.
- However, `lru_cache` caches across *requests*, which may not be desired if the data should refresh. Use with caution.
- Because FastAPI may use multiple worker processes, each will have its own cache.

- *Recommended:* Discussion.

- **Identify inefficiencies in a sample FastAPI endpoint (e.g. missing `async`, no pagination, redundant work).**

- *Difficulty:* Expert.

- *Key Points:*

- Check if the endpoint is not using `async` where it should, causing blocking.
- Look for loading too much data into memory.
- Redundant serialization/deserialization (e.g. calling `jsonable_encoder` multiple times).
- Unnecessary database commits inside loops.
- On the fly data transformations (use database for filtering/sorting when possible).

- *Recommended:* Code review exercise.

- **How would you improve this endpoint to reduce response time? (Given code snippet)**

- *Difficulty:* Expert.

- *Key Points:* (General advice as example)

- Use database indexing on filter fields.
- Cache frequent queries.
- Only select needed columns rather than `SELECT *`.
- Offload heavy computations to background.

- *Recommended:* Discussion (specific code as needed).

- **Spot the bug:**

```
@app.get("/sum")
async def get_sum(numbers: List[int]):
    total = 0
    for n in numbers:
        total += n
    return {"sum": total}
```

- *Difficulty:* Medium.
- *Key Points:*
 - The function signature `async def get_sum(numbers: List[int])` expects the list `numbers` in the *query parameters*, which requires multiple `numbers=` query args (e.g. `?numbers=1&numbers=2`). Perhaps intended to be a body payload? If JSON was intended, must use `Body(...)`.
 - The loop itself is fine; no asynchronous I/O here, so it could be a normal `def`.
 - If `numbers` is large, consider using built-in `sum()` for brevity or use streaming.
- *Recommended:* Code review (catch signature issue, explain query vs body).

Summary of Question Counts by Category

Category	# Questions
Core FastAPI Concepts	8
Pydantic & Data Validation	8
Dependency Injection	7
Async Programming & Concurrency	7
Performance & Benchmarking	8
Security & Auth/Authorization	8
Testing & Debugging	6
Deployment & DevOps (Docker, K8s, CI/CD)	7
Database Integration & ORMs	8
WebSockets & Background Tasks	6
API Design & Versioning	6
Error Handling & Logging	5
Migration & Backward Compatibility	4
Real-World Scenarios / Case Studies	6
Code Review & Optimization	6
Total	100

90-Minute Interview Structure

The following is a suggested allocation of topics for a 90-minute interview with a senior FastAPI candidate:

Section	Time (min)	Topics Covered (sample questions)
Architecture & Core Concepts	10	FastAPI design, request lifecycle, DI basics
Pydantic & Validation	5	Models, validators, response models
Dependency Injection	5	Depends, sub-dependencies, yield cleanup
Async & Concurrency	10	<code>async def</code> vs <code>def</code> , event loop, workers
Performance & Optimization	10	Benchmarking, JSON performance, caching
Security & Auth	10	OAuth2/JWT, API keys, CORS
Database & ORM Integration	10	SQLModel/SQLAlchemy pattern, async ORM, transactions
Deployment & DevOps	10	Docker, workers, CI/CD, scaling
Testing & Debugging	5	TestClient, dependency overrides, logging
WebSockets & Background Tasks	5	WebSocket implementation, background tasks vs Celery
API Design & Versioning	5	Versioning strategies, pagination, versioning
Error Handling & Logging	5	HTTPException, custom handlers, middleware logging
Real-World Scenarios / Code Review	10	Case studies, architecture questions, code critique
Buffer/Follow-up	-	(Extra time as needed)
Total	90	

This structure balances core technical depth (FastAPI specifics, Python async) with system design and devops considerations. Questions can be chosen from the above list according to the candidate's resume and the company's needs. Each question's key points (cited above) guide the expected answer.

Sources: FastAPI official documentation and tutorials [\[4†L277-L286\]](#) [\[6†L482-L490\]](#) [\[21†L579-L588\]](#) [\[22†L61-L69\]](#) [\[24†L328-L332\]](#) [\[34†L242-L245\]](#) [\[46†L339-L347\]](#) [\[51†L781-L784\]](#) [\[53†L557-L560\]](#) [\[55†L639-L645\]](#) [\[60†L826-L833\]](#) [\[66†L320-L327\]](#) [\[69†L25-L34\]](#) [\[73†L43-L51\]](#) [\[82†L898-L902\]](#) [\[85†L815-L823\]](#) [\[88†L1-L4\]](#) [\[92†L295-L300\]](#) [\[94†L324-L332\]](#) (all cited excerpts correspond to the key points above).